

In a Java console application without a database, should a Salary record store the whole Player object or only playerId? Explain the trade-offs and how I can validate that the player exists before adding a salary record.

Thought for a couple of seconds >

A `Salary` record should usually store only `playerId`, not the whole `Player` object.

Storing the whole `Player` object can make it easy to access the player's name or position, but it creates tighter coupling between `Salary` and `Player`. It may also cause confusion when Player information changes later, because an old salary record should still represent the salary details at the time it was created.

Using only `playerId` is cleaner for a console project. `Salary` stores salary-specific data such as `salaryId`, `playerId`, `month`, `year`, `playerMonthlySalary`, `performanceBonus`, and `totalSalary`. When the program needs the player's name or type, it can search for that Player in `PlayerList` using the stored `playerId`.

Before adding a Salary record, validate that the player exists:

```
// Java
Player player = playerList.findPlayerById(salary.getPlayerId());

if (player == null) {
    System.out.println("Player ID does not exist. Please add player first.");
    return;
}
```

After confirming the Player exists, the Salary record can obtain the correct monthly salary from that Player:

```
// Java
salary.setPlayerMonthlySalary(player.calculateMonthlySalary());
salaryRecords.add(salary);
```

This approach avoids duplicating fields such as `fullName`, `age`, `nationality`, `position`, and `shirtNumber` in every Salary record. It also supports one Player having multiple Salary records for different months while keeping the data organized.

📄 🌟 🔍 ↻ ...

```
public class Salary extends ClubRecord {
    private String playerId;

    Player player = playerList.findPlayerById(salary.getPlayerId());
    if (player == null) {
        System.out.println("Player ID does not exist. Please add player first.");
        return;
    }
}
```